CODING CHALLENGE: Geospatial Data Engineer at Corteva Agriscience
Submitted By: Mohammad Wael

Problem 1)
I chose PostgreSQL because it offers the best balance between transactional and analytical workloads. My system requires efficient filtering, aggregation, and idempotent ingestion. PostgreSQL provides native upserts, strong indexing, and partitioning, which makes it a clean and scalable solution without introducing unnecessary complexity.

```sql
CREATE TABLE IF NOT EXISTS weather_observations (
    id BIGSERIAL PRIMARY KEY,
    station_id VARCHAR(20) NOT NULL,
    observation_date DATE NOT NULL,
    max_temp_c NUMERIC(5,2),
    min_temp_c NUMERIC(5,2),
    precipitation_cm NUMERIC(9,3),
    source_file VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    UNIQUE (station_id, observation_date)
);

CREATE INDEX IF NOT EXISTS idx_weather_observations_station_date
    ON weather_observations (station_id, observation_date);

CREATE INDEX IF NOT EXISTS idx_weather_observations_date
    ON weather_observations (observation_date);
```

Problem 2)

- **Ingestion (parsing → DTO → DB):**
  the parser in ingest_weather_file.py (class WeatherStationTextFileParser) reads each station .txt, validates tokens and sentinel values, and builds WeatherObservationCreate DTOs. The ingestion orchestration (WeatherFileIngestor.ingest_file) calls the service ingest_observations_batch to persist records in bulk.
- **Data model / storage:**
  the DB table weather_observations uses a unique constraint on (station_id, observation_date) (see DDL in the repo). ORM models mirror that constraint so each observation is identified by station+date.
- **Idempotent duplicate handling:**
  The repository SQLAlchemyWeatherRepository.bulk_upsert_observations performs chunked dialect-level upserts using INSERT … ON CONFLICT … DO UPDATE (Postgres) or the SQLite equivalent. Because of the unique constraint, repeated runs update the existing row instead of creating duplicates.
- **Accurate inserted vs skipped counts:**
  before each chunk upsert the repository checks how many keys in that chunk already exist, computes inserted = chunk_len - existing_count, commits the chunk, and returns the total inserted count. ingest_file computes skipped = processed - inserted and includes those in the summary and metrics.

- **Aggregation and persistence:** after a successful file ingest WeatherFileIngestor calls WeatherAggregationService.aggregate_year(...), which queries observations (aggregate_yearly_observations), computes averages/totals, builds a WeatherYearlyStatCreate, and upserts it via repository.upsert_yearly_stat.
- **Logging and observability:** process_wx_data.py writes structured logs to ingestion.log (start time, per-file processed/inserted/skipped, summary and duration). The API logging is configured in logger.py to also write structured JSON to logs/api.log. Structured events (start/complete/failure) are emitted with log_structured_event(...) throughout the ingestion flow for metrics and tracing.

Problem 3)

```
CREATE TABLE IF NOT EXISTS weather_yearly_stats (
    id BIGSERIAL PRIMARY KEY,
    station_id VARCHAR(20) NOT NULL,
    year INTEGER NOT NULL,
    avg_max_temp_c NUMERIC(6,3),
    avg_min_temp_c NUMERIC(6,3),
    total_precipitation_cm NUMERIC(12,3),
    observation_count INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    UNIQUE (station_id, year)
);

CREATE INDEX IF NOT EXISTS idx_weather_yearly_stats_station_year
    ON weather_yearly_stats (station_id, year);
```

1. **Trigger**:
   After ingesting a file (in ingest_weather_file.py:134),
   the WeatherFileIngestor.ingest_file() calls WeatherAggregationService.aggregate_year(station, year) for each affected year.
2. **SQL Aggregation** (in weather.py:154):

   ```
   SELECT
     AVG(max_temp_c) as avg_max_temp_c,
     AVG(min_temp_c) as avg_min_temp_c,
     SUM(precipitation_cm) as total_precipitation_cm,
     COUNT(*) as observation_count
   FROM weather_observations
   WHERE station_id = ? AND EXTRACT(YEAR FROM observation_date) = ?
   ```

   The repository method aggregate_yearly_observations() executes this query, computing averages and totals from all observations for that station/year, ignoring NULL values.
3. **Quantization & Validation** (in aggregation.py:66):
   The service receives the aggregate data, quantizes decimals to 2 places using _quantize_decimal(), and builds a WeatherYearlyStatCreate DTO.
4. **Upsert to Database**:
   The service calls repository.upsert_yearly_stat(stat_create), which executes:

   ```
   INSERT INTO weather_yearly_stats (station_id, year, avg_max_temp_c, ...)
   VALUES (...)
   ```

```
ON CONFLICT (station_id, year) DO UPDATE SET ...
```

This ensures yearly stats are always current (inserts new rows or updates existing ones with recalculated values).

Result: After each ingestion run, the weather_yearly_stats table contains aggregated observations automatically synchronized with raw data.

Problem 4)

1. Web Framework: FastAPI

FastAPI is a modern, high-performance Python web framework (built on Starlette and Pydantic) that natively supports async/await, automatic request validation, and built-in OpenAPI/Swagger documentation. It's used for the weather platform API.

2. REST API GET Endpoints with JSON Responses

The API in weather.py provides:

- GET /api/v1/weather/observations — returns paginated observations (PaginatedWeatherObservationRead DTO)
- GET /api/v1/weather/stats — returns paginated yearly statistics (PaginatedWeatherYearlyStatRead DTO)
- GET /api/v1/weather/observations/{station_id}/{observation_date} — returns a single observation
- GET /api/v1/weather/yearly-stats/{station_id} — returns all yearly stats for a station

Each endpoint returns JSON-serialized DTOs with observation/stat fields from the database.

3. Filtering and Pagination via Query Strings

FastAPI's Query dependency injects query parameters into handlers:

```
def query_observations(
    skip: int = Query(0, ge=0, description="Pagination offset"),
    limit: int = Query(100, ge=1, le=1000, description="Page size (max 1000)"),
    station_id: str | None = Query(None, pattern=STATION_ID_PATTERN, description="Optional
station filter"),
    start_date: date | None = Query(None, description="Optional minimum date"),
    end_date: date | None = Query(None, description="Optional maximum date"),
    service: WeatherService = Depends(get_weather_service),
) -> PaginatedWeatherObservationRead:
```

- skip / limit enable offset-based pagination (e.g., ?skip=0&limit=100)
- station_id, start_date, end_date filter results server-side; the service calls the repository query methods
- Query validation (min/max, pattern, type conversion) is automatic via Pydantic; invalid queries return 422

4. OpenAPI / Swagger Documentation

FastAPI auto-generates OpenAPI 3.1.0 spec from route signatures, docstrings, and Pydantic models. The main.py creates the app:

```
app = FastAPI(
    title=settings.app_name,
    version=settings.app_version,
```

```
    docs_url="/docs" if settings.enable_docs else None,
    redoc_url="/redoc" if settings.enable_docs else None,
    openapi_url="/openapi.json" if settings.enable_docs else None,
)
```

- **http://localhost:8000/docs** — interactive Swagger UI with try-it-out capability
- **http://localhost:8000/redoc** — ReDoc documentation view
- **http://localhost:8000/openapi.json** — raw OpenAPI 3.1.0 schema

Each endpoint in weather.py includes docstrings, example responses, and status codes that appear in Swagger automatically.

Problem 5: (Extra Credit)
I would deploy the system using managed AWS services to minimize operational overhead while ensuring scalability and reliability.

For the API, I would containerize the **FastAPI application using Docker** and deploy it on **Amazon ECS with the AWS Fargate launch typ**e. This removes the need to manage servers. The service would be exposed to users **through an Application Load Balancer**, which handles incoming traffic and distributes requests across running tasks.

For the database, I would use **Amazon RDS with PostgreSQL in a Multi-AZ configuration** to ensure high availability and automated failover. Database **credentials and other sensitive configuration would be stored securely in AWS Secrets Manager**.

For scheduled data ingestion, I would use **Amazon EventBridge to define a cron-based schedule** that triggers an ECS Fargate task. This task would run the ingestion code, fetch and process data, and store results in the database and in **Amazon S3 for raw file retention**.

For monitoring and observability, I would rely on **Amazon CloudWatch to collect logs, track metrics, and configure alarms** for failures or performance issues.

Overall, this approach leverages serverless and managed services to provide a scalable, fault-tolerant, and low-maintenance deployment architecture.